

ASSIGNMENT 6

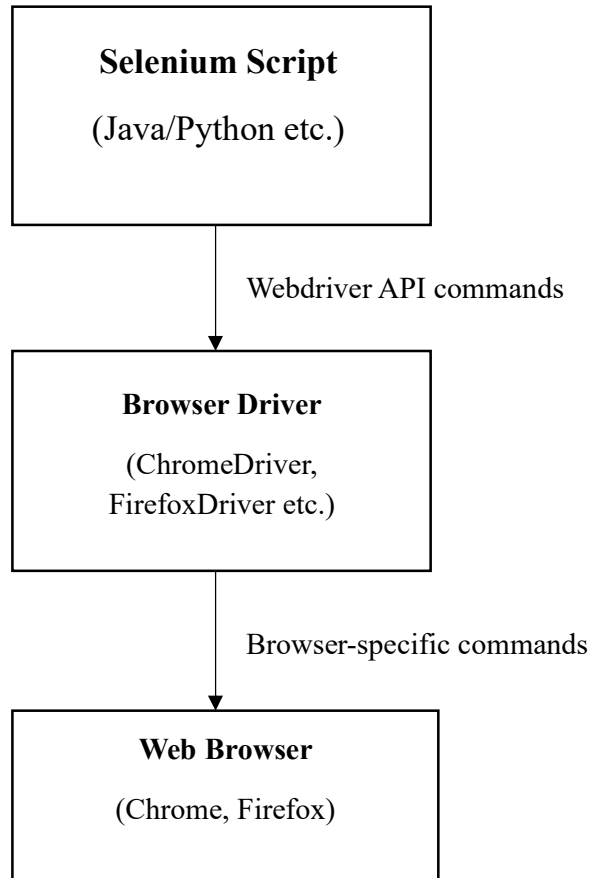
Submitted By: Aiswarya AC

Submitted To: Subin P John

Q1: Explain the WebDriver Architecture in Selenium. Draw and describe how Selenium interacts with the browser through browser-specific drivers.

Selenium WebDriver follows a client-server architecture. It communicates with browsers through browser-specific drivers.

Architecture Diagram:



Working:

1. The Test Script written in Java/Python sends commands through the WebDriver API.
2. The browser-specific driver (ChromeDriver, FirefoxDriver, EdgeDriver) receives these commands.
3. The driver converts the commands into browser-understandable instructions.
4. The browser performs the requested action (click, type, navigate, etc).
5. The browser sends the response back to the driver, which returns it to the test script.

Q2: Write a Selenium WebDriver program in Java to:

Launch Chrome browser

Open the website <https://www.google.com>

Locate the search box using By.name locator

Enter the text Selenium WebDriver and submit

Program:

```
package Assignment6;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class GoogleSearch {

    public static void main(String[] args) {

        WebDriver driver = new ChromeDriver();

        driver.get("https://www.google.com");

        driver.manage().window().maximize();

        WebElement searchBox = driver.findElement(By.name("q"));

        searchBox.sendKeys("Selenium WebDriver");

        searchBox.submit();

    }

}
```

Q3: What are the different locators available in WebDriver? Explain each with an example (e.g., id, name, xpath, cssSelector).

- Locator: **id**
 - Finds an element using its unique ID attribute.
 - Example: `driver.findElement(By.id("username"));`
- Locator: **name**
 - Finds an element using the name attribute.
 - Example: `driver.findElement(By.name("email"));`
- Locator: **cssSelector**
 - Finds elements using CSS syntax.
 - Example: `driver.findElement(By.cssSelector("a[aria-label='kilos']"));`
- Locator: **xpath**
 - Finds elements using XML path expression.
 - Relative xpath:
 - Example 1: `driver.findElement(By.xpath("//input[@id='username']"));`
 - Example 2: `driver.findElement(By.xpath("//label[text()='Address']"));`
 - Example 3: `driver.findElement(By.xpath("//img[contains(@alt, 'INTEL')]"));`
 - Example 4: `driver.findElement(By.xpath("//input[starts-with(@id='user']"));`
 - Absolute xpath:
 - Example: `driver.findElement(By.xpath("/html/body/div/form/input"));`
- Locator: **linkText**
 - Finds a hyperlink using exact visible text.
 - Example: `driver.findElement(By.linkText("Login"));`
- Locator: **partialLinkText**
 - Finds a hyperlink using partial text.
 - Example: `driver.findElement(By.partialLinkText("Log"));`
- Locator: **className**
 - Finds an element using class attribute.
 - Example: `driver.findElement(By.className("loginBtn"));`
- Locator: **tagName**
 - Finds elements using HTML tag name.
 - Example: `driver.findElement(By.tagName("input"));`

Q4: Write a program using Selenium WebDriver to:

Open a demo page

Click a button that triggers a JavaScript alert

Accept the alert

Print “Alert Accepted Successfully” in console

Program:

```
package Assignment6;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class AlertDemo {

    public static void main(String[] args) {

        WebDriver driver = new ChromeDriver();

        driver.get("https://demo.automationtesting.in/Alerts.html");

        driver.manage().window().maximize();

        driver.findElement(By.className("btn-danger")).click();

        driver.switchTo().alert().accept();

        System.out.println("Alert Accepted Successfully");

        driver.quit();

    }

}
```

Output:

Alert Accepted Successfully

Q5: Differentiate between Implicit Wait and Explicit Wait in Selenium. Write a short code snippet showing the use of Explicit Wait to wait until a login button is clickable.

Implicit Wait	Explicit Wait
Applied globally to all elements.	Applied only to specific elements or conditions.
Declared once and remains active throughout the session.	Used whenever required for particular elements.
Waits for a fixed time before throwing an exception.	Waits until a specific condition is met.
Less flexible.	More flexible and efficient.

Program:

```
package Assignment6;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class ExplicitWaitExample {

    public static void main(String[] args) throws InterruptedException {

        ChromeDriver driver = new ChromeDriver();

        driver.get("https://practicetestautomation.com/practice-test-login/");

        driver.manage().window().maximize();
```

```
driver.findElement(By.id("username")).sendKeys("student");  
driver.findElement(By.id("password")).sendKeys("Password123");  
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
        WebElement login =  
wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));  
        login.click();  
        driver.quit();  
    }  
}
```